How to Execute : Mini CPU Simulator in emu8086

This guide covers:

- **Part A**: running the project in emu8086
- **Part B**: demonstrating it step by step (about 5 minutes)
- **Part C**: likely viva questions

Repository: https://github.com/lakshay8toic/Mini-CPU-Simulator

---

Part A : Running the project in emu8086

What you need

- A Windows PC with **emu8086** installed
- The file `src/MINICPU.ASM` from this repository

Step 1 : Download the repository

1. Open **https://github.com/lakshay8toic/Mini-CPU-Simulator**
2. Click the green **Code** button, then **Download ZIP**.
3. Right-click the downloaded ZIP and choose **Extract All**.
4. The program is in `Mini-CPU-Simulator-main\src\MINICPU.ASM`.

Step 2 : Open the file in emu8086

1. Start **emu8086**. Close the welcome or samples window if one appears.
2. Click **Open** (or choose **File > Open**) and select `MINICPU.ASM`.
3. The full source code appears in the editor.

Step 3 : Compile and load

1. Click **Emulate** on the toolbar (or press **F5**).
2. If emu8086 asks where to save `MINICPU.com`, keep the default location and click **Save**.
3. Two windows open: the **emulator** window (registers and memory) and the **original source code** window.

Step 4 : Remove the speed delay (important)

At the bottom of the emulator window, drag the **step delay** slider all the way to the left (**0 ms**). If you skip this, the screen redraws slowly, one character at a time.

Step 5 : Run

1. Click **Run** in the emulator window.
2. The black **emulator screen** window opens and shows the main menu. If it doesn't appear, click the **screen** button in the emulator window.
3. **Click once inside the black screen window** so it receives your key presses.

Step 6 : Check that it works

Press `1`, then any key, then `R`. You should see:

```
 OUTPUT : 0C   (last = 12 decimal)
*** HALT : program finished.  Press any key for the menu ***
```

If you see this, the setup is correct. Press any key to go back to the menu.

Keys used in the simulator

Key	Action
`1` `2` `3`	Load demo program 1, 2 or 3
`4`	Enter your own program in hex
`5`	Show the instruction set
`6` / `Esc`	Exit
`Enter` / `Space`	Run the **next stage** (one clock cycle)
`I`	Run until the current **instruction** completes
`R`	**Run** the whole program up to HALT
`Q`	Return to the menu

Troubleshooting

Problem	Fix
Keys do nothing	Click inside the black emulator screen window first
Screen draws very slowly	Set the step delay slider to 0 ms
The screen window is missing	Click the **screen** button in the emulator window
The program stops in the middle of the demo	Click **Run** again. Don't use **Single step**, which runs one 8086 instruction at a time
You want to start over	Click **Reload** in the emulator window, then **Run**

Part B : Demonstration for the faculty (about 5 minutes)

1. Introduction (15 s)

> "I built a 5-stage processor simulator in 8086 assembly. It runs the Fetch, Decode, ALU, Memory and Write-back model from Unit II, one clock cycle per key press, and shows the RTL action, the control signals and every register at each stage."

2. Instruction set (30 s)

Press `5`. Explain:

- Each instruction is **2 bytes**: opcode, Rdst and Rsrc in byte 0, and Rsrc2, an immediate value or an address in byte 1.
- There are **4 registers**, and **R0 is always 0**.
- There are **11 instructions** using **4 addressing modes**: register, immediate, absolute and index.

Press any key to return to the menu.

3. Demo 1 : Load, Add, Store, stage by stage (2 min)

Press `1`. The listing shows the machine code next to the assembly. Press any key to start.

Press **Enter** five times for the first instruction, `LOAD R1, 30h(R0)`:

Press	Stage	What to say
Enter	**1 FETCH**	"MA gets the PC, IR gets the instruction `64 30`, and the PC goes from 00 to 02."
Enter	**2 DECODE**	"The opcode is LOAD, and RA and RB are read from the register file."
Enter	**3 ALU**	"RZ = 30h. This is the effective address, X + [R0]."
Enter	**4 MEMORY**	"MA = 30h. The value 07 is read from memory into MD, then copied to RY."
Enter	**5 WRITE-BACK**	"R1 = 07. One instruction took 5 clock cycles."

Then press `I` to run one whole instruction at a time:

Press	Instruction	What to point at
I	`LOAD R2, 31h(R0)`	R2 = 05
I	`ADD R3, R1, R2`	R3 = 0C (7 + 5 = 12)
I	`STORE R3, 32h(R0)`	`*0C` appears at address 32h, which is the memory write
I	`OUT R3`	OUTPUT : 0C
I	`HALT`	Clock = **27** (5 instructions × 5 cycles + 2 cycles for HALT)

4. Demo 2 : Loop with a branch (45 s)

Press any key, then `2`, any key, and `R`. The output is `05 04 03 02 01 00`.

> "BEQ decides the branch in the ALU stage. `55 FF` is `ADDI R1, R1, #-1`, with -1 stored in two's complement."

5. Demo 3 : Array sum with index addressing (30 s)

Press any key, then `3`, any key, and `R`. The output is `12` hex = **18**, and `*12` appears at address 3Fh.

> "`LOAD R3, 2Fh(R2)` is index mode, the same as `Load R5, X(R7)` in the slides. The effective address is 2Fh + i."

6. Optional : your own program (30 s)

Press `4` and type `5403` `5804` `1D02` `A300` `.` This program computes 3 + 4 and outputs `07`. It shows that the simulator runs any machine code you type in.

7. Close with the GitHub repository (15 s)

Show the README, `docs/STAGE_TABLES.md` and the `tests` folder, which has 30 automated checks.

Part C : Viva questions and answers

Q1. Why does every instruction take 5 stages? Every instruction uses the same five-stage hardware, and some stages do no work for some instructions. Keeping one organization makes the control unit simple, and it's also what makes pipelining possible (Unit III).

Q2. What are RA, RB, RZ, RM and RY? They are inter-stage registers.

- RA and RB hold the ALU inputs.
- RZ holds the ALU result.
- RM holds the data to be stored.
- RY holds the value to be written back.

They work like the buffers between pipeline stages.

Q3. Is the control hardwired or microprogrammed? Hardwired. The control signals come directly from the opcode and the stage number, which is the step counter.

Q4. Why is R0 hard-wired to 0? So that `X(R0)` gives absolute addressing without needing a separate addressing mode.

Q5. What is the CPI? 5 for every instruction, and 2 for HALT, which stops in the decode stage.

Q6. How is the 8086 used?

- The simulated CPU's memory and registers are stored in the 8086 data segment.
- Instruction fields are decoded with `SHR` and `AND`.
- Arithmetic uses the 8086's own `ADD`, `SUB`, `AND` and `OR`, and the 8086 Z and C flags are copied into the simulated flags.
- The screen uses **INT 21h** (functions 02h and 09h) and **INT 10h**, and the keyboard uses **INT 16h**.

Q7. How do you encode `LOAD R1, 30h(R0)`?

- Opcode 6 = `0110`, Rd = R1 = `01`, Rs = R0 = `00`.
- Byte 0 = `0110 0100` = **64h**.
- Byte 1 = X = **30h**.
- The machine code is **`64 30`**.

Q8. How does this relate to RISC? It follows RISC principles:

- every instruction has a fixed length
- only LOAD and STORE access memory (a load/store architecture)
- the ALU works only on registers
- every instruction goes through the same stages